

Module 2 - Summary Video – Transcript

Note: This transcript is auto-generated from the Teams application; it may include errors.

Hello everyone, I am Maria Konte, the instructor of the Computer Networks course. I would like to welcome you to this video. In this video, we focus on the core ideas that explain what services the transport layer offers on top of the IP protocol and why this matters for different applications.

We are taking a look at the two main transport protocols, UDP and TCP, and how their services are different. And then mainly we are taking a look at how TCP provides reliable, in-order delivery and also how it controls performance using flow control to protect the receiver and congestion control to protect the network, ending with a quick look at the modern and popular TCP variant named CUBIC.

Now, before we dive into the transport layer, let's quickly review the Internet Protocol stack. At the top, we have the application layer that includes protocols such as HTTP and DNS. Below that, we have the transport layer that includes protocols, as we mentioned, TCP and UDP.

And then we have the network layer, which is centered around the IP protocol, which is responsible for moving packets across networks. Then we have the link layer and the physical layer. So the question to ask here is that since we have the IP protocol that is already able to move packets across networks and therefore connect two hosts, then why do we need the transport layer and its protocols?

The answer to that is that IP provides best-effort delivery. In other words, it attempts to deliver the packets, but it does not guarantee delivery, in-order delivery, or reliability. So the transport layer adds an end-to-end service between applications. At the sender, it takes an application message and adds a transport header to form a segment. Then IP encapsulates that segment and carries it across the Internet.

At the receiver side, the transport layer delivers the data to the correct application. Now from here, we will take a look at the two main transport protocols, as we mentioned: UDP, which is a lightweight protocol, and TCP, which offers reliability and control.

Now to make our description of the transport layer more concrete, let's use a running example, which is loading a web page on our browser. Assume that we are using our laptop at our school's network and we type a URL into the browser.

Also, let's assume that the network traffic doesn't go directly to the website server. Instead, it travels from our school network through one or more ISPs and eventually reaches the web server, which is hosted at a content distribution network, placing the content closer to the users.

Now, behind the scenes, our browser needs more than just best-effort packet delivery. It needs the data to arrive correctly and in order and fast enough so that the application feels and is actually responsive for the user.

So here is what happens at a high level. First, our host establishes a TCP connection with the web server, and once that connection is set up, the browser sends an HTTP GET request. The server replies back with an HTTP response, and the body of that response contains the HTML code for the web page. Finally, our browser interprets HTML content and renders the page on our screen.

Now, in this browser example, TCP is the natural choice because the web page must arrive correctly and in order. But not every application needs that level of reliability. Therefore, we develop, for example, let's say that we develop a new application, and then we need to decide what transport service an application needs.

The Internet mainly offers two choices for us, the UDP protocol and the TCP protocol, as we mentioned, and they offer different services to the application above. UDP, on one hand, is lightweight and fast, but it provides only minimal guarantees. It doesn't ensure, for example, reliable delivery or in-order delivery of the packets. On the other hand, TCP is heavier in the sense that it first needs to establish a connection. Overall, it needs more resources from both end hosts, but it provides reliable, in-order delivery and also includes mechanisms for rate control.

So now that we have introduced TCP as the reliable transport option, the next question is how does TCP actually start a connection? So before TCP sends real application data, it first needs to establish a connection using the so-called three-way handshake.

In the first step, the client sends a SYN message, which means that it asks to initiate the connection and establish a connection. The server responds with a SYN-ACK, meaning that they received the request and they are ready for that.

And then finally, the client replies back with an ACK, confirming that the connection is now established. This three-way handshake is important because it synchronizes both sides and establishes the initial sequence numbers. The TCP sequence numbers are essentially byte counters that label the data stream so both hosts can track what has been sent and what has been received. It also signals to both hosts to set aside resources such as buffers, control variables, and other information that will be used to reliably manage the connection throughout.

Now, once communication is complete, then TCP has a process that takes place to shut down the connection. TCP closes connections using a controlled teardown process, so both sides agree that the session is actually finished.

And to prevent cases where one side thinks that the connection is over while the other side still believes that it is active and they are able to send and receive data, TCP closes a connection with a four-step exchange where each side sends a FIN and the other acknowledges it.

So that they both confirm that they are done sending and receiving data and the connection is fully closed. So now that we have seen how TCP sets up and tears down a connection, the next question is what does TCP actually guarantee once the data starts flowing between the two end hosts?

So the key service is reliable transmission. As we mentioned, IP provides best-effort delivery, which means packets may be lost, duplicated, or arrive out of order or late, and that can create problems for different applications. For example, let's say that we are downloading a file and some packets are lost. Then the file can eventually become incomplete or corrupted. UDP leaves that problem to the application to handle, but TCP builds reliability directly into the transport layer, so applications can assume a consistent service.

TCP achieves reliability using feedback, essentially acknowledgments and retransmissions. The receiver sends an acknowledgment, or an ACK as we say, when it successfully receives data. If the sender doesn't receive an acknowledgment within a certain timeout, it assumes loss and retransmits.

This approach is called ARQ, or Automatic Repeat Request. TCP also ensures in-order delivery. Even if packets arrive out of order, TCP is responsible for reordering them before delivering the data stream to the application.

So now that we know that TCP offers reliability as a service because it is able to retransmit lost data, the next question to ask is how does TCP detect loss and also how does it detect it quickly? So we mentioned timeout is one indication that a packet is lost, but if TCP relied only on timeouts, then recovery could be slow, especially if the timeout has a large value. That could create unnecessary delays and slow down performance. Now, to speed this up, TCP uses an additional mechanism, which is called fast retransmit.

The key idea is the following. The receiver sends cumulative acknowledgments for the highest in-order byte that it has received. So if a segment is missing, the receiver keeps sending the same ACK number for every out-of-order segment that it receives.

These repeated acknowledgments are called duplicate acknowledgments. More specifically, looking at the figure in this slide, the sender transmits segments one through 10, but segment 7 is lost in the network. Now, even though segments 8, 9, and 10 still arrive at the receiver, the receiver can't deliver them to the application yet because TCP must deliver the data in order. So the receiver keeps sending acknowledgments that say, I'm still waiting for that segment 7. So that's why we see those three purple acknowledgments labeled with the number 7.

Once the sender receives the third duplicate acknowledgment, it reads that as a signal that segment 7 was lost, not delayed, and therefore it immediately retransmits segment 7 without waiting for a timeout. That retransmission is the fast retransmit shown at the bottom.

So now that we have seen how TCP provides reliability, the next question is how does TCP decide how fast to send data? So this is where transmission control comes in. TCP doesn't just deliver data correctly. It is also responsible for controlling the sending rate to keep communication efficient and stable between the two end hosts. TCP transmission control solves two different problems: flow control and congestion control.

Flow control is responsible for protecting the receiver. Even if the network is fast, the receiving host may have a limited buffer and limited processing speed. So the receiver advertises how much space it has available using a value called the receive window.

The sender then limits how much unacknowledged data it sends so that it doesn't overwhelm the receiver. Now for congestion control, congestion control is responsible for protecting the network, assuming a scenario where we may have multiple senders that perhaps are sharing the same link.

And they may be transmitting too aggressively. Then routers can fill up, queues can build up, delays can increase, and therefore packets can start dropping. TCP detects these signs of congestion and adjusts the sending rate using a value called the congestion window.

So flow control is about the receiver's capacity, and congestion control is about the network's capacity. Now let's take a closer look at flow control. The goal of flow control is to prevent, as we mentioned, the receiver's buffer from overflowing.

In TCP, data can arrive faster than the receiving application can read it. If the application is slow, bytes can build up in memory and may overflow the buffer. In the figure, data arrives from the IP layer into the TCP receive buffer, and the application reads from that buffer at its own pace.

The buffer contains the data currently stored and the remaining free space. The key idea is that the receive window, which represents the available free space, is communicated, or in other words, is advertised to the sender so that the sender knows how much more data it can send safely. If the buffer fills up, then the receive window shrinks and the sender slows down. If space frees up, on the other hand, the receive window grows and the sender is able to speed up again.

Now that we have seen flow control, the next question is how does TCP know when the network is congested? Or in other words, what are the signs of congestion? TCP cannot directly observe congestion inside the routers, but it can infer that from, quote unquote, symptoms.

The two main signals are increasing delays and packet loss. So when router queues start filling up, packets spend more time waiting, so the round-trip time increases. If congestion gets worse and buffers overflow, then packets can also be dropped, often leading to timeouts or duplicate acknowledgments at the sender. TCP treats triple duplicate acknowledgments and timeouts as warnings and adjusts the sending rate to avoid overwhelming the network.

So now that we have seen signs that there is congestion in the network, the next question to ask is how does TCP control congestion in the network? TCP congestion control is designed so that each sender can first probe the available network capacity and then adjust how much data it sends without making congestion worse. TCP uses acknowledgments as feedback, so when acknowledgments come back smoothly, the sender learns that the network can handle more. Now, to do this, TCP maintains a congestion window, which limits how much unacknowledged data can be, as we say, in flight. That means that the data has left the sender, but it has not been acknowledged yet by the receiver. TCP follows a probe-and-adapt approach. In other words, it increases the congestion window when things look stable and decreases the congestion window when it detects congestion. In practice, the sender is limited by both the network and the receiver, so it can only send up to the minimum of the congestion window value and the receiver window value.

Now that we have seen how TCP uses the congestion window to control its sending rate, the next question is how does TCP adjust the congestion window and more specifically, how does it ramp that up when the connection first starts?

So this is where the slow start mechanism comes in. Slow start is the phase where TCP increases its sending rate quickly at the beginning of a connection because it doesn't yet know the network's capacity. It does this by growing the congestion window very rapidly at first.

In this figure, we see the Y axis is the congestion window, the X axis is time, and TCP starts with a small congestion window at first, which is 1 segment during the slow start phase. Then it grows exponentially, doubling in every RTT.

As acknowledgments arrive, that's why the curve rises so sharply. TCP keeps increasing until it reaches a value that is called the slow start threshold, shown by the dashed line. After that, it switches to congestion avoidance. We are going to see that in a second, where it grows more slowly but linearly. And if it detects congestion, then the congestion window drops again and the threshold is updated, and the process repeats.

So now that we have seen how TCP ramps up during the slow start phase, the next question is how does TCP behave after that? So this is where the additive increase multiplicative decrease mechanism comes in.

The basic idea is the following. TCP increases the congestion window when the network looks stable and decreases it when congestion signs appear. So first, TCP grows quickly, as we mentioned, during the slow start phase until it reaches the slow start threshold value.

After that, it enters the congestion avoidance phase, which is shown by the yellow sawtooth pattern on the figure. In this phase, TCP increases the congestion window additively, which is 1 segment per RTT, meaning it grows steadily but not exponentially.

But once TCP detects a congestion signal, specifically a triple duplicate acknowledgment, that is an indication of mild congestion. And when that happens, it drops the congestion window size to half of what it was before.

Then TCP starts increasing again, probing to see how much capacity is available in the network, and that repeated increase and cutback behavior is exactly what creates the sawtooth shape that we see on the figure.

Now this figure summarizes how the TCP congestion window changes over time. As we mentioned, it starts with slow start, when the congestion window grows quickly. After it reaches the threshold, TCP enters congestion avoidance, where the congestion window increases more gradually in a sawtooth pattern.

And the important part here is how TCP reacts to two different congestion signals. So if TCP observes triple duplicate acknowledgments, then it treats that as mild congestion, as an indication of mild congestion, and therefore cuts the size of the congestion window in half and then continues increasing again.

But if TCP detects a timeout, then this is considered a severe indication of congestion, so the congestion window drops more aggressively back to the initial value, and TCP essentially restarts.

Finally, we are taking a look at how one of the most popular TCP versions, namely TCP CUBIC, adjusts the congestion window efficiently on high-speed networks. So CUBIC still increases the congestion window when the network is stable and backs off when it detects losses.

But what is different here is how that congestion window growth is happening. So instead of growing strictly 1 packet per RTT, CUBIC uses a cubic function of time since the last loss. After a loss, it reduces the congestion window.

And it remembers the previous maximum value, then grows slowly as it approaches that old maximum value and faster once it passes it to probe for more bandwidth. Because this growth depends on time rather than RTT, CUBIC is also more RTT fair, meaning that flows with different RTTs compete more evenly when using the same link.

In summary, in this video, we explained why the transport layer is needed on top of IP's best-effort delivery. We briefly compared the transport layer protocols, UDP and TCP. We described how TCP provides reliable, in-order communication using acknowledgments, retransmissions, and fast retransmit specifically. Finally, we covered how TCP controls its sending rate with flow control and congestion control using mechanisms such as slow start, additive increase, multiplicative decrease, and the modern TCP version, namely TCP CUBIC.

Thank you for watching!